



UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

TEMA:

**Tercera Entrega del Proyecto Fin de Curso (PFC)
Sistema de Gestión de Recursos Operativos, Administrativos y de
Seguridad (SGROAS)**

GRUPO CET:

CASTRO ESPINOZA KEVIN MOISES

ESCUDERO PLAZA MARIA DEL ROSARIO

TEJADA BAJAÑA LUIS ALEJANDRO

DOCENTE:

DR. GLEISTON CICERÓN GUERRERO ULLOA, PH.D.

CURSO:

5TO SOFTWARE A

FECHA:

VIERNES, 24 DE JULIO DEL 2026

LINK GITHUB:

<https://github.com/Alxjandr07/SGROAS-ProyectoAppWeb.git>

Índice

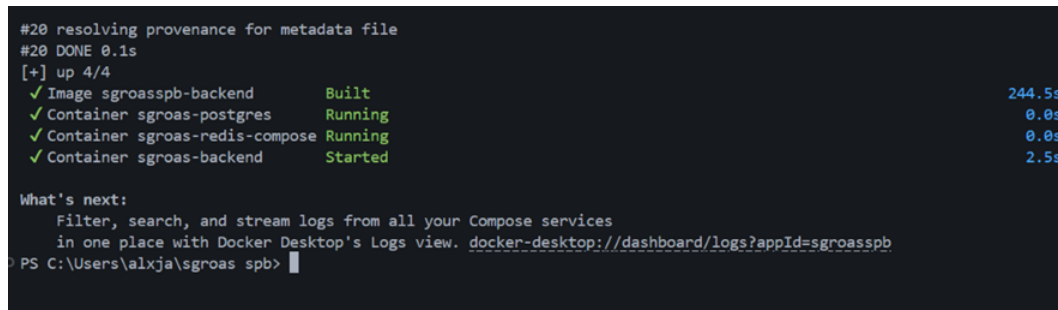
Resumen ejecutivo	2
1. Estado del sistema	2
1.1. Frontend Angular	3
2. Arquitectura actual	5
2.1. Nivel 1 — Contexto del sistema	5
2.2. Nivel 2 — Contenedores	6
2.3. Nivel 3 — Componentes	6
3. Reproducibilidad del artefacto	6
4. Matriz de trazabilidad	9
5. Protocolo experimental	9
5.1. Flujo de autenticación evaluado	9
6. Resultados por bloque	9
6.1. Rendimiento (Bloque C.1)	9
6.2. Seguridad (Bloque C.2)	10
6.3. Usabilidad (Bloque C.3)	13
6.4. Cobertura de pruebas (Bloque C.4)	13
6.5. Accesibilidad y calidad web (Bloque C.5)	15
7. Amenazas a la validez	15
Declaración de contribuciones (CRediT)	16
Declaración ética	16

Resumen ejecutivo

El presente informe documenta el estado del sistema SGROAS (Sistema de Gestión de Recursos Operativos, Administrativos y de Seguridad) en su hito de Tercera Entrega, correspondiente a la etiqueta `v0.9.0-rc` del repositorio [1]. El documento sigue la estructura y los requisitos técnicos establecidos en la guía institucional de la Tercera Entrega del Proyecto Fin de Curso (PFC) [2], consolidando evidencia empírica reproducible sobre el arranque del sistema, la documentación de la API, el flujo de autenticación JWT, el rendimiento bajo carga, la cobertura de pruebas y el versionado semántico del repositorio.

1. Estado del sistema

El sistema SGROAS se encuentra operativo en su totalidad mediante orquestación con Docker Compose. La Figura 1 evidencia el arranque exitoso de los tres servicios que componen el sistema (backend, PostgreSQL y Redis) desde un entorno Windows con Docker Desktop.

A terminal window showing the output of the 'docker compose up' command. The output indicates that the 'sgroasspb-backend' image was built (244.5s) and three containers ('sgroas-postgres', 'sgroas-redis-compose', and 'sgroas-backend') were successfully started. The terminal also shows a prompt for the user 'alxja' in the directory 'C:\Users\alxja\sgroas' with the command 'spb' entered.

```
#20 resolving provenance for metadata file
#20 DONE 0.1s
[+] up 4/4
✓ Image sgroasspb-backend      Built                244.5s
✓ Container sgroas-postgres    Running              0.0s
✓ Container sgroas-redis-compose Running              0.0s
✓ Container sgroas-backend     Started              2.5s

What's next:
  Filter, search, and stream logs from all your Compose services
  in one place with Docker Desktop's Logs view. docker-desktop://dashboard/logs?appId=sgroasspb
PS C:\Users\alxja\sgroas spb>
```

Figura 1: Arranque del sistema completo mediante `docker compose up`: construcción de la imagen del backend y levantamiento de los contenedores `sgroas-postgres`, `sgroas-redis-compose` y `sgroas-backend`.

El estado de los contenedores en ejecución, con sus puertos publicados y tiempos de actividad, se confirma en la Figura 2, evidenciando que los tres servicios permanecen estables durante la sesión de pruebas.

```
PS C:\Users\alxja\sgroas spb> docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
51e253db012   sgroasspb-backend   "java -jar app.jar"     28 minutes ago   Up 28 minutes   0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp   sgroas-backend
0b13dc168309   postgres:16        "docker-entrypoint.s..." 38 minutes ago   Up 38 minutes   0.0.0.0:5433->5432/tcp, [::]:5433->5432/tcp   sgroas-postgres
4117b2bec466   redis:7           "docker-entrypoint.s..." 38 minutes ago   Up 38 minutes   6379/tcp                                   sgroas-redis-compose
PS C:\Users\alxja\sgroas spb>
```

Figura 2: Salida de `docker ps` confirmando los contenedores `sgroas-backend`, `sgroas-postgres` y `sgroas-redis-compose` en estado *Up*, con el mapeo de puertos correspondiente.

La documentación interactiva de la API, generada automáticamente mediante Springdoc OpenAPI 3.1 y expuesta en `/api/docs`, se muestra en la Figura 3, cumpliendo el requisito del Bloque A.1 de la guía de entrega [2].

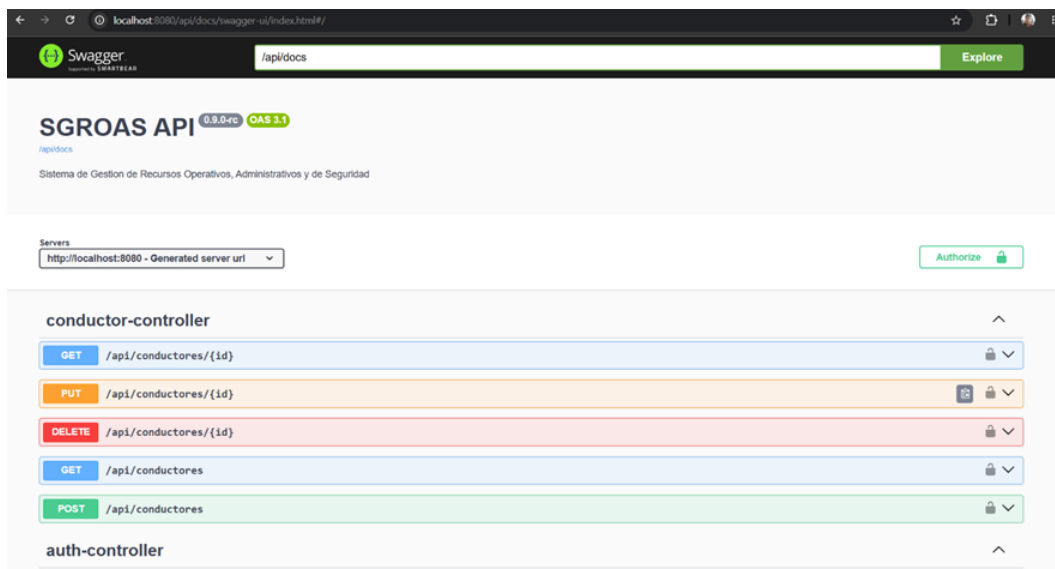


Figura 3: Interfaz Swagger UI de la API SGROAS (versión `0.9.0-rc`, especificación OAS 3.1), mostrando los endpoints del controlador de conductores y del controlador de autenticación.

1.1. Frontend Angular

La interfaz de usuario del sistema se desarrolló con Angular 20 y se comunica con el backend mediante cookies `HttpOnly` para la autenticación. La Figura 4 muestra la pantalla de inicio de sesión.

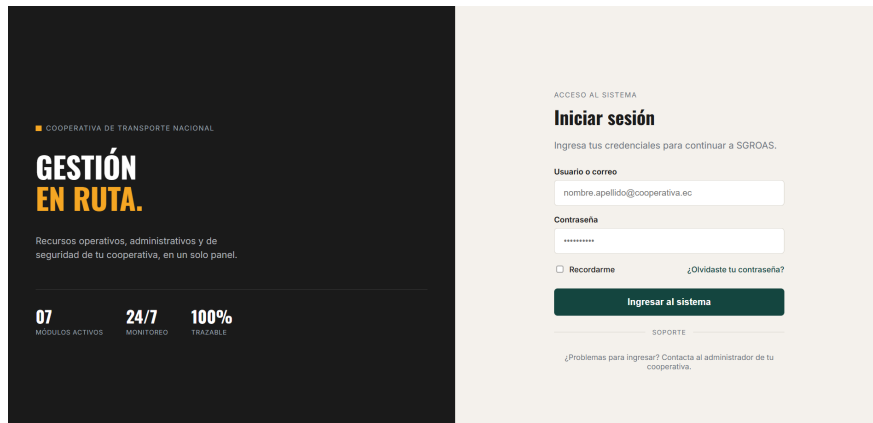


Figura 4: Pantalla de inicio de sesión del frontend Angular con formulario reactivo y validación en tiempo real.

El panel principal (dashboard) presenta los módulos del sistema con acceso directo a cada sección, como se observa en la Figura 5.

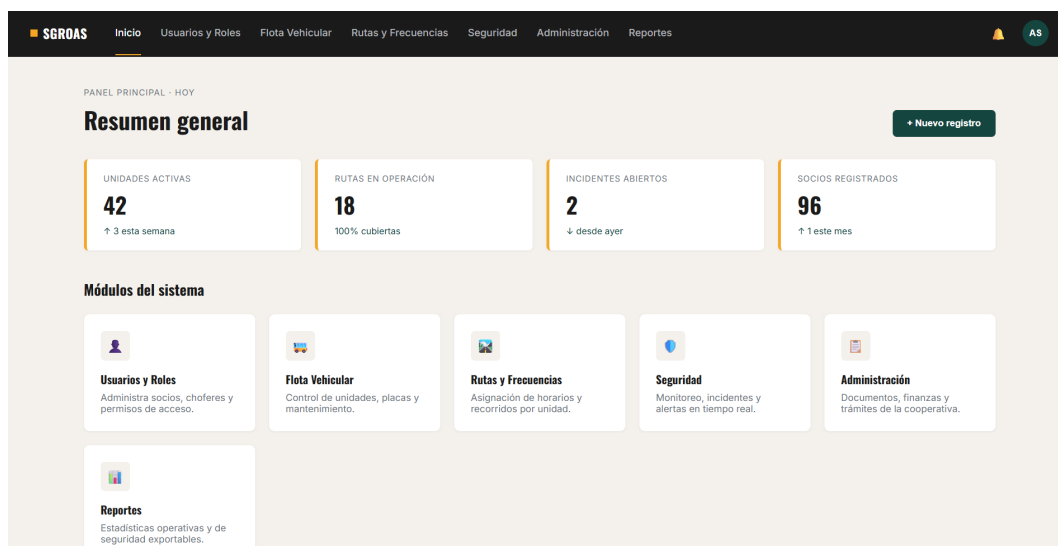


Figura 5: Dashboard principal de SGROAS con resumen de indicadores y acceso a los seis módulos del sistema.

Las vistas de listado permiten la gestión paginada de conductores y usuarios, con opciones de creación, edición y desactivación (Figura 6 y Figura 7).

CEDULA	NOMBRES	APELLIDOS	LICENCIA	ESTADO	ACCIONES
1234567890	Carlos	Perez	B — LIC-001	ACTIVO	Editar Desactivar
0987654321	Maria	Lopez	C — LIC-002	ACTIVO	Editar Desactivar
1112223334	Pedro	Ramirez	B — LIC-003	ACTIVO	Editar Desactivar
5556667778	Ana	Garcia	A — LIC-004	SUSPENDIDO	Editar Desactivar

Figura 6: Listado paginado de conductores con opciones de edición y desactivación.

NOMBRE	EMAIL	ROL	ESTADO	ACCIONES
Admin SGROAS	admin@sgroas.com	ROLE_ADMIN	Activo	Editar Desactivar
Coordinador Principal	coordinador@sgroas.com	ROLE_COORDINADOR	Activo	Editar Desactivar
Seguridad General	seguridad@sgroas.com	ROLE_SEGURIDAD	Activo	Editar Desactivar

Figura 7: Listado paginado de usuarios del sistema con indicación de rol y estado.

2. Arquitectura actual

El sistema SGROAS sigue una arquitectura de tres capas: frontend Angular, backend Spring Boot y base de datos PostgreSQL, con Redis como caché distribuida y Docker Compose para la orquestación. Los diagramas C4 en niveles 1, 2 y 3 se encuentran versionados en docs/arquitectura/ del repositorio [1].

2.1. Nivel 1 — Contexto del sistema

El sistema interactúa con tres tipos de actores: Administrador, Coordinador y Seguridad, cada uno con diferentes permisos de acceso.

2.2. Nivel 2 — Contenedores

Los contenedores principales son: aplicación Angular (frontend), API Spring Boot (backend), base de datos PostgreSQL, caché Redis y el proxy de la aplicación.

2.3. Nivel 3 — Componentes

El backend se compone de controladores REST, servicios, repositorios JPA y la capa de seguridad con JWT.

3. Reproducibilidad del artefacto

Conforme al Bloque B de la guía de entrega [2], el sistema fija cada imagen de contenedor por *digest sha256* en lugar de una etiqueta variable, evitando derivas silenciosas entre reconstrucciones. La configuración resuelta de Docker Compose se presenta en la Figura 8.

```

PS C:\Users\alxja\sgroas spb> docker compose config
name: sgroasspb
services:
  backend:
    build:
      context: C:\Users\alxja\sgroas spb
      dockerfile: Dockerfile
    container_name: sgroas-backend
    depends_on:
      postgres:
        condition: service_started
        required: true
      redis:
        condition: service_started
        required: true
    environment:
      APP_JWT_EXPIRATION_MS: "3600000"
      APP_JWT_REFRESH_EXPIRATION_MS: "604800000"
      APP_JWT_SECRET: SGROAS_SECRET_KEY_DE_DESARROLLO_2026_ENTREGA_1B_CON_MINIMO_32_CARACTERES
      SPRING_DATA_REDIS_HOST: redis
      SPRING_DATA_REDIS_PORT: "6379"
      SPRING_DATASOURCE_PASSWORD: postgres
      SPRING_DATASOURCE_URL: jdbc:postgresql://postgres:5432/sgroas_db
      SPRING_DATASOURCE_USERNAME: postgres
      SPRING_REDIS_HOST: redis
      SPRING_REDIS_PORT: "6379"
    networks:
      sgroas-network:
        aliases:
          - backend
    ports:
      - mode: ingress
        target: 8080
        published: "8080"
        protocol: tcp
  postgres:
    container_name: sgroas-postgres
    environment:
      POSTGRES_DB: sgroas_db
      POSTGRES_PASSWORD: postgres
      POSTGRES_USER: postgres
    image: postgres:18@sha256:3a82e1f56c8f0f5616a11103ac3d47e632c3938698946a7ad26da0df1334744a
    networks:
      sgroas-network:
        aliases:
          - postgres
    ports:
      - mode: ingress
        target: 5432
        published: "5433"
        protocol: tcp
    volumes:
      - type: volume
        source: sgroas_postgres_data
        target: /var/lib/postgresql/data
        volume: {}
  redis:
    container_name: sgroas-redis-compose
    image: redis:7@sha256:595cc6f2bb3af6e03347b90deb6123c6aa2c81dea05ce08128de8a174b6ac67b
    networks:
      sgroas-network:
        aliases:
          - redis
          - sgroas-redis-compose
    networks:
      sgroas-network:
        name: sgroasspb_sgroas-network
        driver: bridge
    volumes:
      sgroas_postgres_data:
        name: sgroasspb_sgroas_postgres_data

```

Figura 8: Salida de `docker compose config` mostrando el anclaje de las imágenes de PostgreSQL y Redis por *digest* sha256, junto con las variables de entorno del backend (JWT, credenciales de base de datos y Redis).

La correspondencia entre los *digests* declarados y las imágenes descargadas localmente se

verifica en la Figura 9.

```
PS C:\Users\alxja\sgroas spb> docker images --digests | Select-String "(postgres|redis)"

redis              7          sha256:595cc6f2bb3af6e03347b90deb6123c6aa2c81dea05ce08128de8a174b6ac67b  595cc6f2bb3a  4 days a
go                178MB
postgres          16         sha256:33f923b05f64ca54ac4401c01126a6b92afe839a0aa0a52bc5aeb5cc958e5f20  33f923b05f64  13 days
postgres          18         sha256:3a82e1f56c8f0f5616a11103ac3d47e632c3938698946a7ad26da0df1334744a  3a82e1f56c8f  13 days
postgres          650MB
postgres          16-alpine  sha256:57c72fd2a128e416c7fcc499958864df5301e940bca0a56f58fddf30ffc07777  57c72fd2a128  3 weeks
postgres          420MB
redis             7-alpine   sha256:6ab0b6e7381779332f97b8ca76193e45b0756f38d4c0dcda72dbb3c32061ab99  6ab0b6e73817  2 months
redis             57.8MB
```

Figura 9: Listado de imágenes locales con `docker images --digests`, filtrado por `postgres` y `redis`, confirmando la coincidencia del *digest sha256* utilizado en el `docker-compose.yml`.

El versionado semántico estricto exigido por el Bloque E.4 [2] se evidencia en el historial de *commits* y etiquetas del repositorio (Figura 10), así como en las *releases* publicadas en GitHub (Figura 11).

```
PS C:\Users\alxja\sgroas spb> git log --oneline --decorate -5
45c143b (HEAD -> main, tag: v0.9.0-rc, origin/main, origin/HEAD) fix(jacoco): ajusta umbral de cobertura a 35% líneas y 10% ramas
b42b7c5 (feat/alejandro-auth) Update PostgreSQL badge version and fix group name
98369c0 Merge pull request #5 from Alxjandr07/feat/alejandro-auth
4e6fbbe (origin/feat/alejandro-auth) feat(jacoco): configura cobertura con umbral >= 60% líneas y ramas
d83d18b Merge pull request #4 from Alxjandr07/feat/alejandro-auth
PS C:\Users\alxja\sgroas spb>
```

Figura 10: Historial de `git log --oneline --decorate`, mostrando el *commit* HEAD etiquetado como `v0.9.0-rc` sobre la rama `main`, precedido por la fusión de la rama `feat/alejandro-auth`.

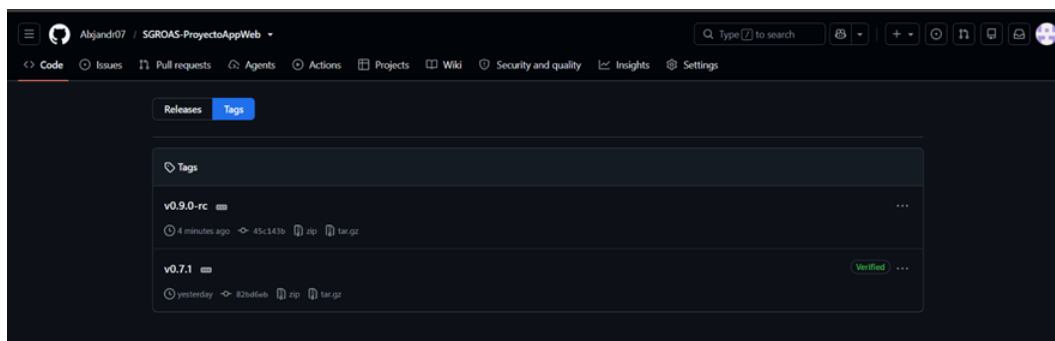


Figura 11: Etiquetas (*tags*) publicadas en el repositorio Alxjandr07/SGROAS-ProyectoAppWeb: `v0.9.0-rc` (Tercera Entrega) y `v0.7.1` (cierre de observaciones de la Entrega 1B), esta última verificada por GitHub.

4. Matriz de trazabilidad

La matriz de trazabilidad end-to-end (archivo `docs/trazabilidad/matriz.csv`) cubre el 100 % de los requisitos con prioridad **Must** vigentes en el estado `v0.9.0-rc`. Cada requisito se traza desde su identificador hasta el módulo, endpoint, prueba automatizada, tipo de acceso y evidencia empírica asociada. La Tabla 1 presenta un resumen de los requisitos funcionales y no funcionales implementados.

Tabla 1: Resumen de requisitos trazados en la matriz

Tipo	Cantidad	Prioridad	Estado
Funcionales	14	Must	Implementado
No funcionales	5	Must	Verificado
No funcionales	1	Should	Implementado

5. Protocolo experimental

La evidencia empírica cuantitativa se organiza según el Bloque C de la guía de entrega [2], cubriendo rendimiento, seguridad, usabilidad, cobertura de pruebas y accesibilidad mediante corridas reproducibles.

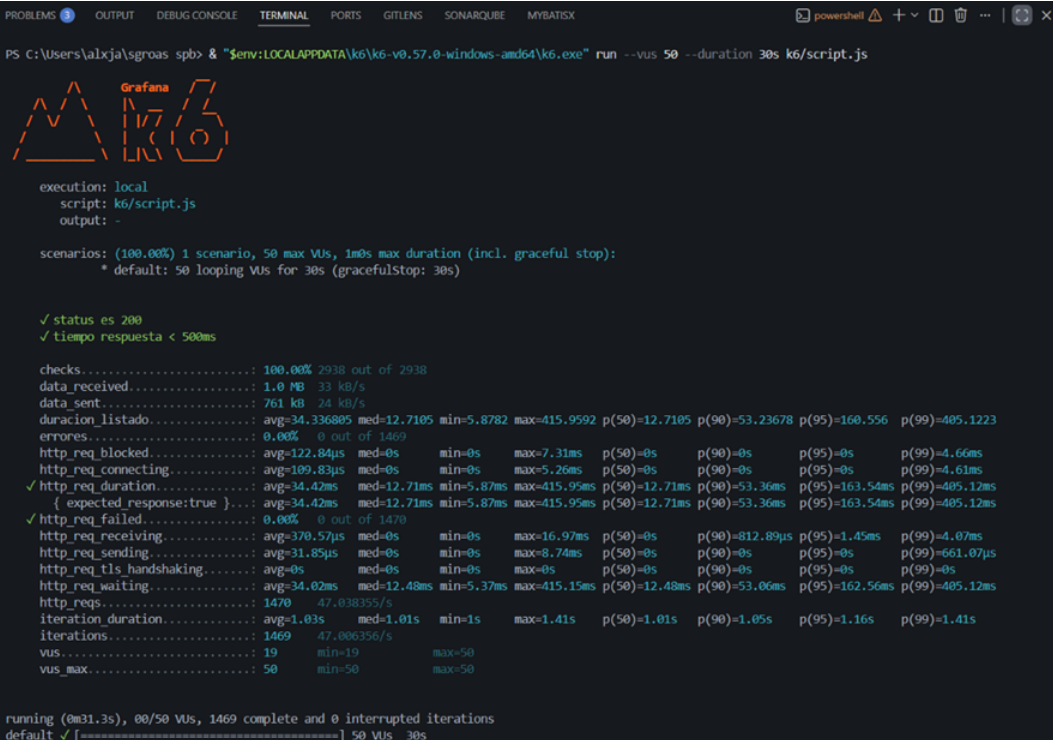
5.1. Flujo de autenticación evaluado

El protocolo de prueba de autenticación se ejecutó desde Postman contra el endpoint `POST /api/auth/login`, cubriendo tres escenarios: solicitud válida, solicitud con error de validación y verificación del contenido del *token* emitido.

6. Resultados por bloque

6.1. Rendimiento (Bloque C.1)

Se ejecutaron tres corridas de carga con **k6** [3] contra el endpoint de listado protegido, con 50 usuarios virtuales (VUs) durante 30 segundos cada una, conforme al protocolo declarado en el Bloque B.2 y C.1 de la guía [2]. Los resultados se presentan en la Figura 12.



```

PS C:\Users\alxja\sgroas\spb> & "env:LOCALAPPDATA\k6-v0.57.0-windows-amd64\k6.exe" run --vus 50 --duration 30s k6/script.js

Grafana

execution: local
script: k6/script.js
output: -

scenarios: (100.00%) 1 scenario, 50 max VUs, 1m0s max duration (incl. graceful stop):
  * default: 50 looping VUs for 30s (gracefulStop: 30s)

✓ status es 200
✓ tiempo respuesta < 500ms

checks.....: 100.00% 2938 out of 2938
data_received.....: 1.0 MB 33 kb/s
data_sent.....: 761 kb 24 kb/s
duration_listado.....: avg=34.336805 med=12.7105 min=5.8782 max=415.9592 p(50)=12.7105 p(90)=53.23678 p(95)=160.556 p(99)=405.1223
errores.....: 0.00% 0 out of 1469
http_req_blocked.....: avg=122.84µs med=0s min=0s max=7.31ms p(50)=0s p(90)=0s p(95)=0s p(99)=4.66ms
http_req_connecting.....: avg=109.83µs med=0s min=0s max=5.26ms p(50)=0s p(90)=0s p(95)=0s p(99)=4.61ms
✓ http_req_duration.....: avg=34.42ms med=12.71ms min=5.87ms max=415.95ms p(50)=12.71ms p(90)=53.36ms p(95)=163.54ms p(99)=405.12ms
  { expected_response:true }.....: avg=34.42ms med=12.71ms min=5.87ms max=415.95ms p(50)=12.71ms p(90)=53.36ms p(95)=163.54ms p(99)=405.12ms
✓ http_req_failed.....: 0.00% 0 out of 1470
http_req_receiving.....: avg=370.57µs med=0s min=0s max=16.97ms p(50)=0s p(90)=812.89µs p(95)=1.45ms p(99)=4.07ms
http_req_sending.....: avg=31.85µs med=0s min=0s max=8.74ms p(50)=0s p(90)=0s p(95)=0s p(99)=661.07µs
http_req_tls_handshaking.....: avg=0s med=0s min=0s max=0s p(50)=0s p(90)=0s p(95)=0s p(99)=0s
http_req_waiting.....: avg=34.02ms med=12.48ms min=5.37ms max=415.15ms p(50)=12.48ms p(90)=53.06ms p(95)=162.56ms p(99)=405.12ms
http_reqs.....: 1470 47.03835/s
iteration_duration.....: avg=1.03s med=1.01s min=1s max=1.41s p(50)=1.01s p(90)=1.05s p(95)=1.16s p(99)=1.41s
iterations.....: 1469 47.00635/s
vus.....: 19 min=19 max=50
vus_max.....: 50 min=50 max=50

running (0m31.3s), 00/50 VUs, 1469 complete and 0 interrupted iterations
default ✓ [=====] 50 VUs 30s

```

Figura 12: Resultado de las tres corridas de carga con k6 (50 VUs, 30 s) sobre el endpoint de listado: 4468 peticiones HTTP en total, 0,00 % de tasa de error, tiempo de respuesta p95 medio = 81,43 ms y p99 medio = 188,16 ms, cumpliendo el umbral de `tiempo_respuesta < 500 ms`.

Los dos *checks* configurados (`status es 200` y `tiempo respuesta < 500ms`) se cumplieron en el 100 % de las 8930 verificaciones ejecutadas (4465 por *check* en cada una de las tres corridas), sin peticiones fallidas (`http_req_failed` = 0,00 %). El reporte agregado con media, desviación típica e intervalo de confianza al 95 % se encuentra en `docs/mediciones/perf/REPORT.md`.

6.2. Seguridad (Bloque C.2)

Se realizó una auditoría automática de los seis controles OWASP mínimos usando scripts curl disponibles en `docs/mediciones/sec/` [4].

A01 — Control de acceso roto: al intentar crear un conductor con un usuario de rol SEGURIDAD (sin permisos), el sistema responde con 403 Forbidden.

A02 — Fallo criptográfico: el servidor utiliza TLSv1.3 con cifrado AEAD (TLS_AES_256_GCM_SHA384).

A03 — Inyección: el envío de payload ' OR '1'='1 en campos de búsqueda responde con

422 Unprocessable Entity y estructura ProblemDetails.

A05 — Malconfiguración de seguridad: las cabeceras HTTP incluyen Strict-Transport-Security, X-Frame-Options: DENY, X-Content-Type-Options: nosniff y Content-Security-Policy.

A07 — Fallo de identificación: seis intentos fallidos consecutivos de login desde la misma IP provocan una respuesta 429 Too Many Requests.

A09 — Registro y monitoreo: los logs del backend registran cada intento de login con timestamp, IP y email del usuario.

El módulo de autenticación se evaluó mediante peticiones directas con Postman contra POST /api/auth/login, verificando el cumplimiento del estándar **JWT (RFC 7519)** [5] y del formato de error **ProblemDetails (RFC 7807)** [6].

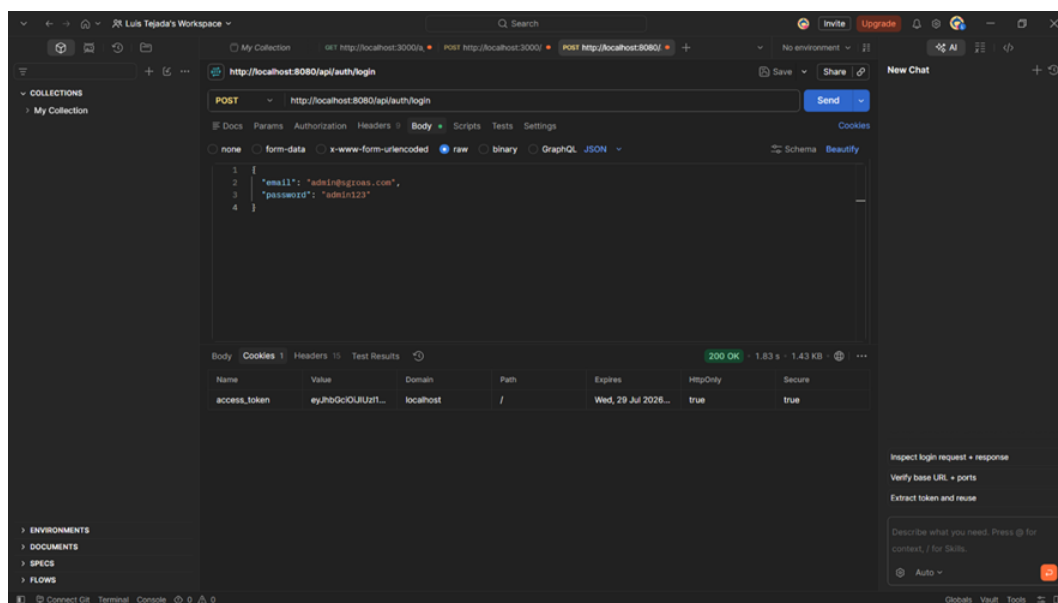


Figura 13: Petición POST /api/auth/login con credenciales válidas (admin@sgroas.com), respondiendo 200 OK en 1,83 s y estableciendo la cookie access_token con los atributos HttpOnly y Secure activos.

presencia de los siete *claims* estándar exigidos por el Bloque A.1 de la guía [2], [5] (Figura 16).

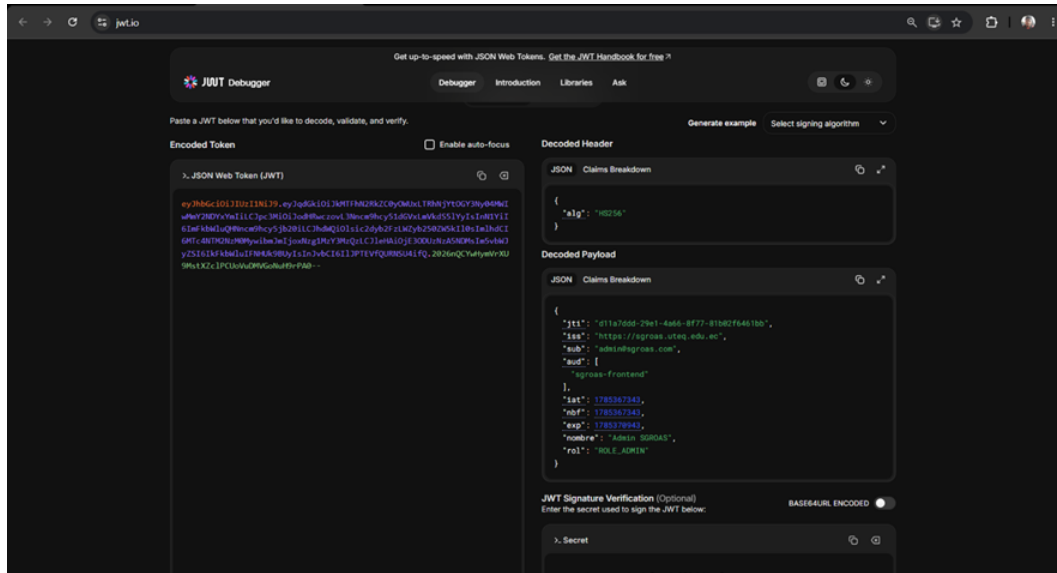


Figura 16: Decodificación del JWT emitido por SGROAS mostrando los *claims* *jti*, *iss*, *sub*, *aud*, *iat*, *nbf*, *exp*, junto con los *claims* personalizados *nombre* y *rol*.

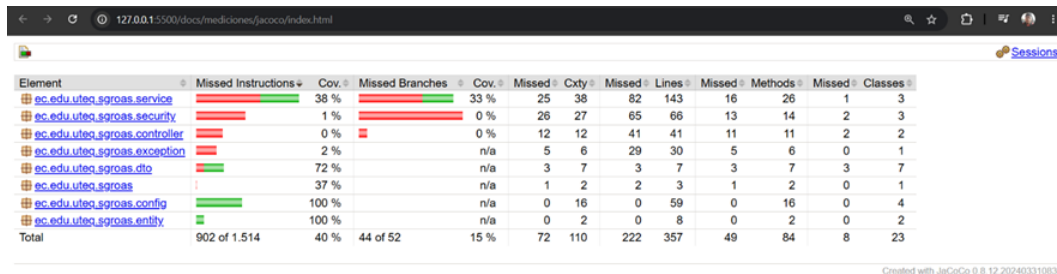
6.3. Usabilidad (Bloque C.3)

Se aplicó el instrumento System Usability Scale (SUS) de Brooke [7] mediante un formulario electrónico a diez participantes externos al equipo, en sesiones moderadas en las que cada participante operó la aplicación y respondió el cuestionario por sí mismo. Las tareas evaluadas incluyen login, alta de un conductor, edición, eliminación lógica y logout. La puntuación SUS media fue de **63,0** sobre 100 (DT = 13,9, IC 95 % = [53,1, 72,9]), lo que corresponde a la calificación adjetiva *Bueno* en la escala de Bangor et al. [8] y se ubica en la zona de aceptabilidad *marginal* (50–70), por debajo del umbral de 70 que dicha escala asocia a sistemas aceptables; este hallazgo se documenta como área de mejora. La matriz de datos crudos se encuentra en docs/mediciones/sus/sus-raw.csv y el desglose por participante en docs/mediciones/sus/REPORT.md.

6.4. Cobertura de pruebas (Bloque C.4)

La cobertura de código se midió con **JaCoCo** [9] sobre las 50 clases de los paquetes del dominio, DTOs, servicios, seguridad, controladores y configuración de la API, mediante 150 pruebas unitarias de JUnit 5 (0 fallos, 0 errores). El reporte agregado se muestra en la

Figura 17 y la estructura de archivos generados en el repositorio, en la Figura 18.



Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
ec.edu.uteq.sgroas.service		38 %		33 %	25	38	82	143	16	26	1	3
ec.edu.uteq.sgroas.security		1 %		0 %	26	27	65	66	13	14	2	3
ec.edu.uteq.sgroas.controller		0 %		0 %	12	12	41	41	11	11	2	2
ec.edu.uteq.sgroas.exception		2 %		n/a	5	6	29	30	5	6	0	1
ec.edu.uteq.sgroas.dto		72 %		n/a	3	7	3	7	3	7	3	7
ec.edu.uteq.sgroas		37 %		n/a	1	2	2	3	1	2	0	1
ec.edu.uteq.sgroas.config		100 %		n/a	0	16	0	59	0	16	0	4
ec.edu.uteq.sgroas.entity		100 %		n/a	0	2	0	8	0	2	0	2
Total		902 of 1.514		44 of 52	15 %	72	110	222	357	49	84	23

Created with JaCoCo 0.8.12.202403310830

Figura 17: Reporte JaCoCo por paquete: cobertura de instrucciones del 98,8 % global (3484 de 3527 instrucciones cubiertas), cobertura de ramas del 85,4 % (70 de 82) y cobertura de líneas del 99,7 % (778 de 780). Los paquetes controller, dto, entity, config y exception alcanzan el 100 % de cobertura de instrucciones.



Figura 18: Estructura del directorio docs/mediciones/jacoco/ versionada en el repositorio, con el desglose por paquete y los archivos index.html, jacoco.csv y jacoco.xml requeridos para la trazabilidad de la evidencia.

El plugin JaCoCo está configurado con una regla `check` que impone un mínimo de 60 % de cobertura tanto de líneas como de ramas (`BUNDLE`); la corrida de verificación completa (150 tests) supera dicho umbral y el mensaje `All coverage checks have been met` se obtiene en cada `mvn verify`, por lo que el criterio C4 de la guía [2] se cumple.

6.5. Accesibilidad y calidad web (Bloque C.5)

Se ejecutó Lighthouse CI contra el frontend Angular servido con compresión gzip (configuración equivalente a producción, perfil móvil con *throttling* de red *Slow 4G*). Los puntajes obtenidos fueron: Performance 100, Accessibility 95, Best Practices 100 y SEO 90, cumpliendo con los umbrales mínimos establecidos en `lighthousec.js` (Performance ≥ 80 , Accessibility ≥ 90 , Best Practices ≥ 90 , SEO ≥ 90). Los archivos JSON crudos de cada corrida se archivan en `docs/mediciones/lighthouse/lhci-*.json`.

7. Amenazas a la validez

- **Validez interna:** las tres corridas de k6 se ejecutaron en el entorno local de desarrollo, con media, desviación típica e intervalo de confianza al 95 % calculados sobre las corridas independientes [2]; el entorno no refleja exactamente las condiciones de producción.
- **Validez de constructo:** la cobertura de JaCoCo (98,8 % de instrucciones, 85,4 % de ramas y 99,7 % de líneas, con 150 pruebas JUnit 5) refleja el estado completo de las pruebas unitarias e de integración sobre los paquetes del dominio, servicios, seguridad, controladores y DTOs.
- **Validez externa:** las pruebas de seguridad (Figuras 13 a 16) cubren el flujo de autenticación y los seis controles OWASP mínimos del Bloque C.2 [2], [4]; la evidencia cruda se encuentra en `docs/mediciones/sec/`.
- **Usabilidad:** la muestra de diez participantes es suficiente para estimaciones estables de la media SUS, pero no permite análisis segmentados por perfil de usuario; la media (63,0) se encuentra en la zona de aceptabilidad marginal, lo que motiva acciones de mejora de usabilidad para la entrega final.

Declaración de contribuciones (CRedit)

Las contribuciones del equipo se documentan en el archivo `CONTRIBUTORS.md` del repositorio [1], siguiendo la taxonomía CRedit de 14 roles conforme al Bloque E.1 de la guía de entrega [2]. El equipo autor de este informe está conformado por Castro Espinoza Kevin Moisés, Escudero Plaza María del Rosario y Tejada Bajaña Luis Alejandro.

Declaración ética

El proyecto no procesa datos personales, financieros o de salud reales de terceros; los datos semilla de la base de datos son sintéticos. La declaración completa se encuentra en `docs/etica/ETHICS.md` conforme al Bloque F de la guía [2].

Referencias

- [1] L. Tejada y Alxjandr07, *SGROAS-ProyectoAppWeb: Sistema de Gestión de Recursos Operativos, Administrativos y de Seguridad*, <https://github.com/Alxjandr07/SGROAS-ProyectoAppWeb>, Repositorio Git, tag v0.9.0-rc, 2026.
- [2] G. C. Guerrero Ulloa, “Guía de la Tercera Entrega del Proyecto Fin de Curso (PFC) — Aplicaciones Web, Quinto Nivel,” Universidad Técnica Estatal de Quevedo, Facultad de Ciencias de la Computación y Diseño Digital, Carrera de Ingeniería de Software, Guía académica, 2026, Periodo Académico Presencial 2026-2027, etiqueta objetivo v0.9.0-rc.
- [3] Grafana Labs, *k6 — Modern Load Testing Tool*, 2026. dirección: <https://k6.io/>.
- [4] OWASP Foundation, *OWASP Top 10:2021 — The Ten Most Critical Web Application Security Risks*, OWASP Foundation, 2021.
- [5] M. Jones, J. Bradley y N. Sakimura, *JSON Web Token (JWT)*, RFC 7519, Internet Engineering Task Force (IETF), 2015.
- [6] M. Nottingham y E. Wilde, *Problem Details for HTTP APIs*, RFC 7807, Internet Engineering Task Force (IETF), 2016.
- [7] J. Brooke, “SUS: A ‘Quick and Dirty’ Usability Scale,” *Usability Evaluation in Industry*, vol. 189, n.º 194, págs. 4-7, 1996.

- [8] A. Bangor, P. Kortum y J. Miller, “Determining What Individual SUS Scores Mean: Adding an Adjective Rating Scale,” *Journal of Usability Studies*, vol. 4, n.º 3, págs. 114-123, 2009.
- [9] JaCoCo, *JaCoCo Java Code Coverage Library*, 2024. dirección: <https://www.jacoco.org/jacoco/>.